ROS 2 BASICS: NODES + COMMUNICATION

Setup • Workspace • Packages • ROS Graph • Topics/Services/Actions • Talker/Listener

UOttawa Mobile Robotics • Instructor: Ali Karimzadeh

uOttawa

ROS 2 in 60 seconds

ROS 2 helps you build robot software as many small programs that communicate.

- **Node** = a running program (a process).
- **Interfaces** = message definitions (msg/srv/action).
- **Communication** = Topics / Services / Actions.
- ROS tools help you **see the graph** and debug logically.
- Docs hub: [ROS 2 Humble Docs]

Terminal Setup: the must-do steps

Every new terminal must know ROS 2 paths (we call this “sourcing”).

- Source ROS 2 (Humble):
- ```
  source /opt/ros/humble/setup.bash
  ```
- Quick check:
- ```
  printenv ROS_DISTRO
  which ros2
  ros2 node list
  ```
- Source: [Configuring ROS 2 Environment]

Create one workspace: `~/ros2_ws`

Workspace = where you create packages and build them with `colcon`.

- Create folders:
- ```
  mkdir -p ~/ros2_ws/src
  ```
- Build (even empty, creates install/):
- ```
  cd ~/ros2_ws
  colcon build --symlink-install
  ```
- Source overlay (your workspace):
- ```
  source ~/ros2_ws/install/setup.bash
  ```
- Source: [Creating a workspace]

uOttawa
  ```

What is a package?

Package = the build + share unit in ROS (code + metadata + dependencies).

- A package contains: code, `package.xml`, `setup.py` (Python) or CMake files.
- ROS finds executables through the package install + entry points.
- We use **Python packages** today: `ament_python` + `rclpy`.
- Official: [Creating a package]

uOttawa

Create our class package: `ros_basics`

This package will contain our **talker** and **listener** nodes.

- Commands:
- ```
  cd ~/ros2_ws/src
  ros2 pkg create --build-type ament_python ros_basics --dependencies rclpy std_msgs
  ```
- Check it exists:
- ```
  ls ~/ros2_ws/src
  ```
- Docs: [ros2 pkg create]

uOttawa

Package structure (where code goes)

Python nodes go inside the inner module folder: `ros_basics/ros_basics/`

- Go to the package root (this folder contains metadata + build files):
- ```
  cd ~/ros2_ws/src/ros_basics
  ls
  ```
- **`package.xml`** → package metadata + **dependencies** (ROS uses this to know what you need).
- **`setup.py`** → Python install + **entry points** (`ros2 run ...` comes from here).
- **`resource/ros_basics`** → small marker file so ROS can discover the package name (ament indexing).
- **`ros_basics/` (inner folder)** → the **Python module**; put your nodes here (`talker.py`, `listener.py`).
- **`ros_basics/__init__.py`** → makes the folder a Python package (enables `import ros_basics`).
- (Sometimes) **`launch/`** → launch files (we will use later, not today).
- (Sometimes) **`config/`** → YAML configs (parameters, tuning values).

uOttawa
  ```

What is a node?

Node = a running process that performs one job and communicates.

- Examples: `camera_node`, `lidar_driver`, `nav2_controller`, `talker`.
- A node usually has: **publishers**, **subscriptions**, timers, services, actions.
- ROS tools can list nodes and show what they publish/subscribe.
- CLI: Understanding nodes

Quiz: Package vs Node

Which statement is correct?

A Package is a running process; node is a folder.

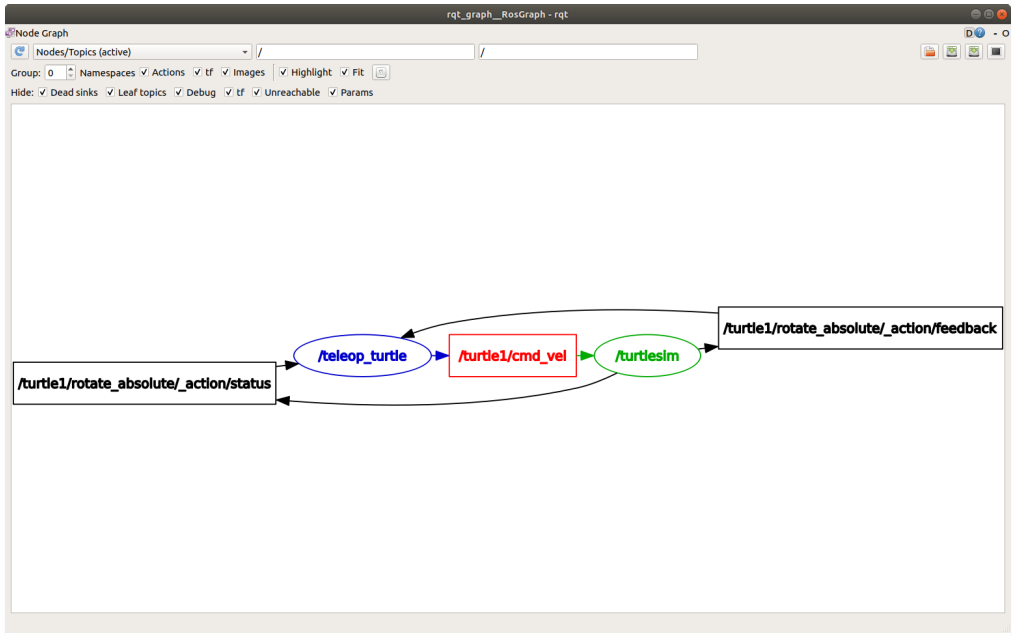
B Node is a running process; package is the build/share folder unit.

C Package and node mean the same thing in ROS 2.

D Node is only for C++; packages are only for Python.

The ROS Graph (visual map of your system)

- The ROS graph shows **nodes** and how they connect.
- `rqt_graph` draws pub/sub connections (very useful in labs).
- We use it to answer: “Who publishes this topic?”
- Docs: [Understanding Topics (rqt_graph)]



Source: ROS 2 docs (rqt_graph.png)

Communication types (how nodes talk)

In ROS 2, nodes communicate through **interfaces** (msg/srv/action).

- **Topics** → data stream (publish/subscribe).
- **Services** → quick request/response (call-and-reply).
- **Actions** → long task (goal + feedback + result + cancel).
- Rule of thumb: **stream → topic**, **ask → service**, **long task → action**.
- Docs: Topics, Services, Actions

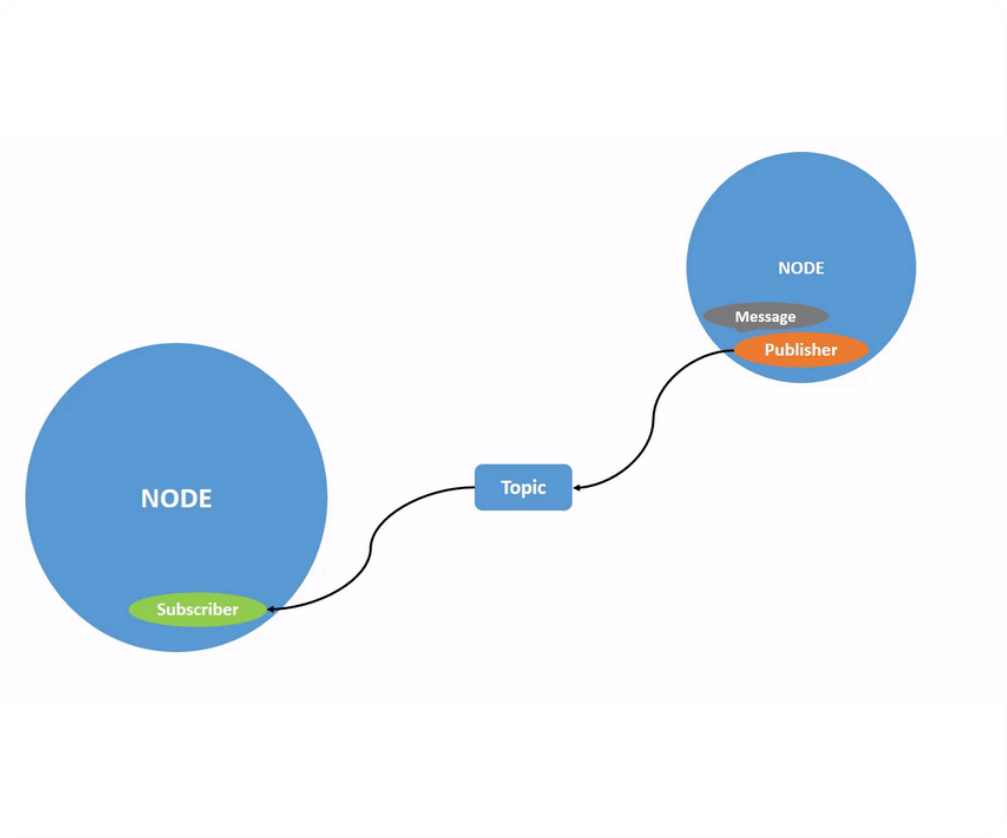
Topics (Pub/Sub) — what you must know

Topics are the most common communication type in robotics.

- Publisher sends messages to a named channel like `/chatter` or `/cmd_vel`.
- Subscribers receive messages **whenever they arrive** (callbacks).
- Publishers and subscribers do **not** need to know each other directly.
- **Message type must match** (example: `std_msgs/msg/String`).
- Docs: [Understanding Topics]

Topics = Publish/Subscribe (data stream)

- **Many-to-many** (many publishers and subscribers can exist).
- Best for continuous data: sensors, state, velocity commands.
- CLI tools you'll use:
-
```
ros2 topic list
ros2 topic echo /chatter
ros2 topic info /chatter
```
- Source: [Understanding topics]



Source: ROS 2 docs (Topic pub/sub gif)

uOttawa

Services — when and why

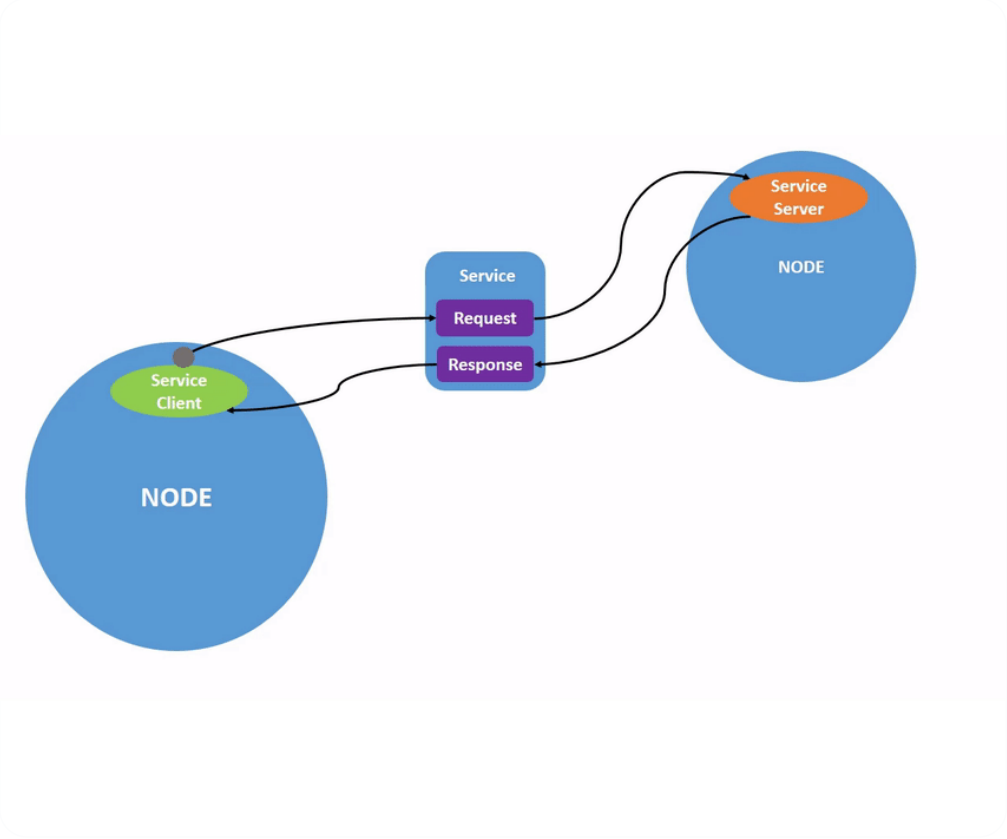
Services are for quick “ask and get an answer” tasks.

- Client sends a request → server replies with a response.
- Best when you need a **single result** now (not a stream).
- Common examples: reset, clear costmap, trigger something once.
- CLI examples:
- ```
  ros2 service list
  ros2 service type /clear
  ```
- Docs: [Understanding Services]

uOttawa
  ```

Services = Request/Response (call-and-reply)

- Client sends a **request**, server returns a **response**.
- Use when you need an answer **now** (fast).
- CLI tools:
-
```
ros2 service list
ros2 service type /clear
ros2 interface show std_srvs/srv/Empty
```
- Source: [Understanding services]



Source: ROS 2 docs (Service request/response gif)

uOttawa

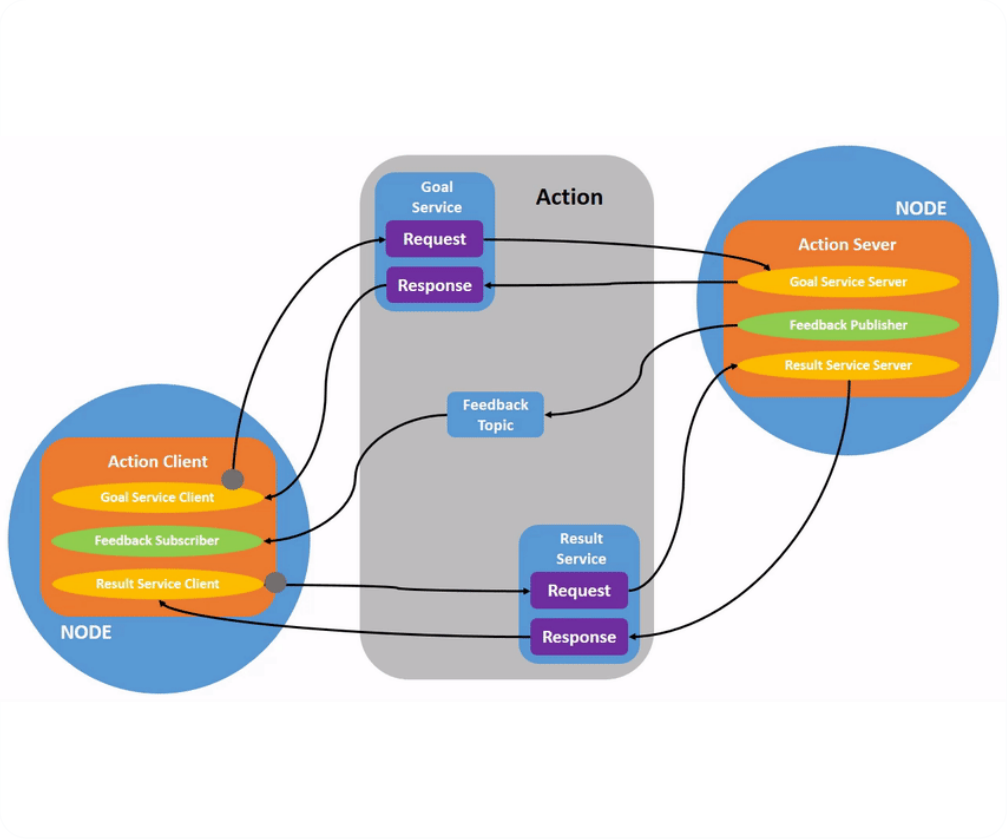
Actions — when and why

Actions are for tasks that take time and need feedback/cancel.

- Client sends a **goal** → server sends **feedback** → final **result**.
- Actions can be **canceled** (important for robots).
- Common examples: navigation goal, docking, long manipulation tasks.
- Docs: [Understanding Actions]

Actions = Goal + Feedback + Result (+ Cancel)

- For **long-running** tasks (seconds/minutes).
- Client sends a **goal** → server sends **feedback** → final **result**.
- Actions are **cancelable**.
- Source: [Understanding actions]



Source: ROS 2 docs (Action goal/feedback/result gif)

uOttawa

ROS 2 Interfaces (Msg / Srv / Action)

Interfaces are the **data contracts** that define what is sent between nodes.

- **Message (`.msg`)** → used by **Topics** (one-way stream). It defines the fields inside each message.
- **Service (`.srv`)** → used by **Services** (request/response). It has **Request** fields and **Response** fields.
- **Action (`.action`)** → used by **Actions** (goal/feedback/result). It defines **Goal**, **Result**, and **Feedback**.
- You can inspect any interface on your system with these CLI commands:
-
```
ros2 interface list
ros2 interface show std_msgs/msg/String
ros2 interface show example_interfaces/srv/AddTwoInts
ros2 interface show example_interfaces/action/Fibonacci
```
- **Key idea:** the interface file defines **what data exists** (types + field names) — your node code only **fills/reads** those fields.
- Docs: [Understanding interfaces]

uOttawa

Quiz: Pick the right communication

You want to continuously publish wheel velocities at 20 Hz. What should you use?

- **A** Topic
- **B** Service
- **C** Action
- **D** Parameter

Exercise: Talker + Listener (Python pub/sub)

We will create two nodes inside our `ros_basics` package.

- **talker** publishes String messages on topic `chatter`.
- **listener** subscribes to `chatter` and prints received messages.
- We will then inspect the graph with `ros2 node` and `ros2 topic`.
- Official tutorial (full version): [Py Publisher/Subscriber]

Step 1 — Create the node files

Create files inside: `~/ros2_ws/src/ros_basics/ros_basics/`

- Commands:
- ```
  cd ~/ros2_ws/src/ros_basics
  ls
  ```
- Create files:
- ```
  touch ros_basics/talker.py
  touch ros_basics/listener.py
  ```
- Open for editing:
- ```
  nano ros_basics/talker.py
  ```


uOttawa
  ```

Step 2 — `talker.py` (Part 1: imports + node class)

This is the minimum skeleton: imports + Node class.

- Paste this into `talker.py`:
-
```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class Talker(Node):
    pass
```
- **Why these lines?** `rclpy` = ROS2 Python; `Node` = base class; `String` = message type.

uOttawa

`talker.py` (Part 2: `__init__` = publisher + timer)

In `__init__`, we create a publisher and a timer callback.

- Replace `pass` with:
- ```
    def __init__(self):
        super().__init__('talker')
        self.pub = self.create_publisher(String, 'chatter', 10)
        self.timer = self.create_timer(0.5, self.on_timer)
        self.i = 0
```
- **Explanation:** `'talker'` = node name • `'chatter'` = topic • `10` = queue depth • `0.5s` = 2 Hz timer.



uOttawa
```

`talker.py` (Part 3: publish in timer callback)

Every timer tick → create msg → publish → log.

- Add this method under `__init__`:
-
```
    def on_timer(self):
        msg = String()
        msg.data = f'Hello ROS2 {self.i}'
        self.pub.publish(msg)
        self.get_logger().info(f'Publishing: "{msg.data}"')
        self.i += 1
```
- **Key line:** `self.pub.publish(msg)` sends the message on `chatter`.

uOttawa

`talker.py` (Part 4: `main()` + `spin()`)

`spin()` keeps the node alive and processes callbacks (timers/subscriptions).

- Add at the bottom of file:
-
```
def main():
    rclpy.init()
    node = Talker()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```
- **Why spin?** Without spin, the program would exit immediately.

uOttawa

Step 3 — `listener.py` (Part 1: imports + class)

Subscriber node skeleton.

- Open file:
- ```
  nano ros_basics/listener.py
  ```
- Paste:
- ```
  import rclpy
  from rclpy.node import Node
  from std_msgs.msg import String

  class Listener(Node):
      pass
  ```
- **Same imports** as talker because we use same message type.

uOttawa

`listener.py` (Part 2: subscribe + callback)

Subscription calls your callback each time a message arrives.

- Replace `pass`:
-
```
    def __init__(self):
        super().__init__('listener')
        self.sub = self.create_subscription(
            String, 'chatter', self.cb, 10
        )

    def cb(self, msg: String):
        self.get_logger().info(f'I heard: "{msg.data}"')
```
- **Important:** topic name `'chatter'` must match talker.

uOttawa

`listener.py` (Part 3: main)

Subscription calls your callback each time a message arrives.

- add main:
-
```
def main():
    rclpy.init()
    node = Listener()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```


uOttawa

Step 4 — Register executables in `setup.py`

This makes `ros2 run ros_basics talker` work.

- Open `setup.py`:
- ```
  nano ~/ros2_ws/src/ros_basics/setup.py
  ```
- Find `entry_points` and set:
- ```
  entry_points={
    'console_scripts': [
      'talker = ros_basics.talker:main',
      'listener = ros_basics.listener:main',
    ],
  },
  ```
- Format: `name = package.module:function`

uOttawa

Step 5 — Build + source your workspace

After adding code, you must build and source again.

- Commands:
-
```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
```
- Why `--symlink-install`? Edits to Python update faster during development.

Step 6 — Run (2 terminals)

Terminal A: listener first. Terminal B: talker.

- Terminal A:
- ```
  source /opt/ros/humble/setup.bash
  source ~/ros2_ws/install/setup.bash
  ros2 run ros_basics listener
  ```
- Terminal B:
- ```
  source /opt/ros/humble/setup.bash
  source ~/ros2_ws/install/setup.bash
  ros2 run ros_basics talker
  ```


uOttawa

Step 7 — Inspect with ROS CLI (what's happening?)

These commands confirm the system is connected correctly.

- List nodes and topics:
- ```
  ros2 node list
  ros2 topic list
  ```
- Check topic details:
- ```
  ros2 topic info /chatter
  ros2 topic echo /chatter
  ```
- Optional graph GUI:
- ```
  rqt_graph
  ```


uOttawa
  ```

Quiz: What does the `10` mean?

In `create_publisher(..., 10)` or `create_subscription(..., 10)`, the `10` is best described as...

- **A** The topic publishing frequency (10 Hz)
- **B** A queue depth (buffer size) for recent messages
- **C** The message size limit (10 bytes)
- **D** The number of nodes allowed to subscribe

uOttawa

Optional: Save your work with Git (local)

Make a **repo folder in** `src/` and put your packages inside.

- Create repo folder:
- ```
  mkdir -p ~/ros2_ws/src/my_ros_repo
  cd ~/ros2_ws/src/my_ros_repo
  ```
- Create packages inside it:
- ```
  ros2 pkg create --build-type ament_python ros_basics --dependencies rclpy std_msgs
  ```
- Initialize Git + first commit:
- ```
  git init
  git add .
  git commit -m "Initial commit"
  ```
  ```

Optional: Push to GitHub (submission link)

If you submit a GitHub link, the repo should include your package + a short README.

- Create a new repo on GitHub (website) → copy the repo URL.
- Add remote and push (HTTPS example):
- ```
  git remote add origin https://github.com/<user>/<repo>.git
  git branch -M main
  git push -u origin main
  ```
- If you update later:
- ```
  git add .
  git commit -m "Update"
  git push
  ```
- Source: GitHub — Adding a remote • GitHub — Push commits

Assignment: ROS 2 Action (Goal → Feedback → Result)

For your final project you must send goal positions, so next session you will practice Actions.

- **Task:** Build a **simple Action Server + Action Client**.
- **Behavior required:**
- • Client sends a goal with **x, y, z-orientation** (any random values).
- • Server immediately responds: **"Got it"** (goal accepted).
- • After **5 seconds** → publish feedback: **"Halfway there"**.
- • After another **5 seconds** → send result: **"Reached goal"**.
- **Submission:**
- • A **short video** showing it working on your computer.
- • Submit **either** a ZIP of the package **or** a **GitHub repository link**.
- Helpful links: Understanding Actions • Writing an Action Server/Client (Python)

Wrap-up + Next Topics

Today you learned the **core mental model** of ROS 2.

- ✅ Workspace + package creation
- ✅ Node vs package vs ROS graph
- ✅ Topics / Services / Actions (what to use when)
- ✅ Python pub/sub talker + listener built and running
- **Next topics (we will teach next):**
- • **TF2 + TF Tree** (frames like `map → odom → base_link`)
- • **URDF** (robot description used by RViz, TF, simulation)
- • **Actions Assignment** (needed for final projects to send goal positions)
- Preview links: TF2 intro • URDF • Actions

uOttawa